Network Flows

Pritha Banerjee

University of Calcutta

banerjee.pritha74@gmail.com

September 19, 2025

Overview

1. Preliminaries
2. Ford-Fulkerson Method
3. Analysis of Ford-Fulkerson Method
4. Maximum Flow (Max Flow) and Minimum Cut(Min Cut)
5. Maximum Bipartite Matching : A reduction to Maximum Flow Problem
6. Extension of Network Flows
   - Circulation
   - Circulation with upper and lower capacity bounds
   - Application: Survey design
7. Books

Table of Contents

1. Preliminaries
2. Ford-Fulkerson Method
3. Analysis of Ford-Fulkerson Method
4. Maximum Flow (Max Flow) and Minimum Cut(Min Cut)
5. Maximum Bipartite Matching : A reduction to Maximum Flow Problem
6. Extension of Network Flows
   - Circulation
   - Circulation with upper and lower capacity bounds
   - Application: Survey design
7. Books

Network Flows: Definitions (1)

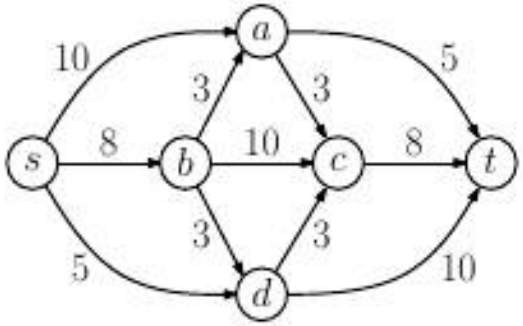
Network flow is the name of a variety of related graph optimization problems.

- A **flow network** is a **directed graph** $G = (V, E)$ in which each edge $(u, v) \in E$ has a nonegative capacity $c(u, v) \geq 0$. If $(u, v) \notin E$ set $c(u, v) = 0$.
- There are two **special vertices:** a **source** s, and a **sink** t; assume that there is no edge entering s and no edge leaving t; so, flow network is also called $s - t$ *network*
- Assume that every vertex lies on some path from the source s to the sink t. This implies that $m \geq n - 1$, where $n = |V|$ and $m = |E|$
- All capacities $c(u, v)$ are integers; if not can make them integer by multiplying them with l.c.m of denominators.

Applications:

- Edges could be thought of as "pipes", capable of carrying some "stuff" of any measurable quantity such as fluid, megabytes of network traffic, commodities, currency.
- The idea is to find out **how much flow we can push** from a designated **source node to a designated sink node**.

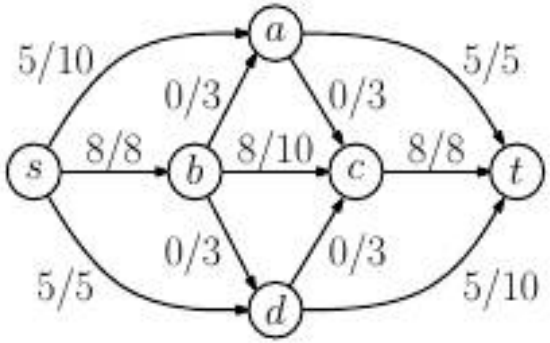
Network Flow: Definitions(2)



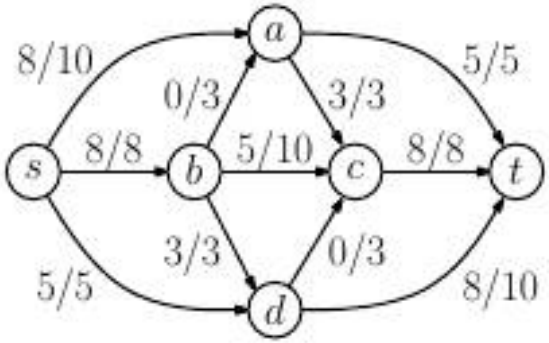
Flows, Capacities, and Conservation:

- Given an $s-t$ network, a **flow** (also called an $s-t$ flow) is a function f that maps each edge to a **non-negative** real number and satisfies the following properties:
  - **Capacity Constraint:** $\forall (u,v) \in E$, $f(u,v) \leq c(u,v)$.
  - **Flow conservation (or flow balance):** $\forall v \in V \backslash \{s,t\}$, the sum of flow along edges into v ($f^{in}(v) = \sum_{(u,v)\in E} f(u,v)$) equals the sum of flows along edges out of v ($f^{out}(v) = \sum_{(v,w)\in E} f(v,w)$); that is $f^{in}(v) = f^{out}(v), \forall v \in V \backslash \{s,t\}$
- The value of a **flow** f, denoted $|f|$, is **sum of flows out of** s, $|f| = f^{out}(s) = \sum_{(s,w)\in E} f(s,w)$; amount of flow that will reach from s to t.

Network Flow: Definitions(3)



A valid flow ($|f| = 18$)



A maximum flow ($|f| = 21$)

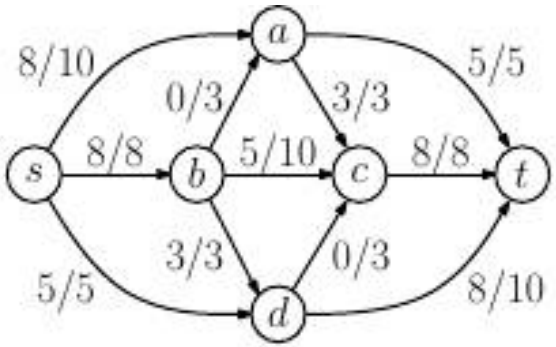
Note: on every edge flow / capacity is written in figure; Left: $5 + 8 + 5 = 18$, Right: $8 + 8 + 5 = 21$

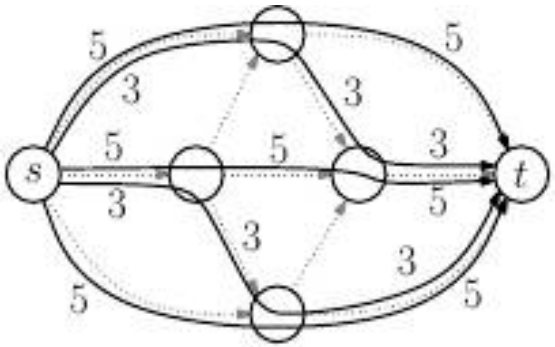
Maximum Flow:

- Given an $s - t$ network, the optimization problem is to determine a **flow** of **maximum value**.
- **Problem Statement:** Given a flow network $G = (V, E)$, and source and sink vertices s and t, find the flow of maximum value from s to t; flow of value $8 + 8 + 5 = 21$, is maximum for this network.
- Although the value of the **maximum flow** is **unique**, there may be many different **flow functions** that achieve this maximum value.

Path-based flow Vs. Edge-based flow

- **Edge-based flow:** Flow conservation and capacity constraints have to be satisfied
- **Path-based flow:** It is a function that assigns each $s - t$ path a **non-negative** real number such that, for every edge $(u, v) \in E$, the sum of the flows on all the paths containing this edge is **at most** $c(u, v)$; flow conservation constraint not required because in each path the flow into a vertex (excluding s and t), carries an equivalent amount of flow out of that vertex.





Claim: Given an $s - t$ network G, no edges entering s or leaving t, G has an **edge-based flow** of value x if and only if G has a **path-based flow** of value x.

Variants of Flow Network

- **Multi-source, multi-sink networks:** many source vertices $s_1, \cdots, s_k$ and many sink vertices $t_1, \cdots, t_l$. This can be modeled by just adding a special super-source s_0 and a super-sink t_0 , and attaching s_0 to all the s_i and attach all the t_j to t_0. Let these edges have infinite capacity. Now by pushing the maximum flow from s_0 to t_0 the maximum flow from all the s_i's to all the t_j's is obtained.
- **Multicommodity flow problem:** When the flow from a certain source must arrive at a designated sink vertex. For example, when sources are manufacturing production centers, sinks are retail outlets, and the amount of commodity from s_i to arrive at t_j is to be determined. An NP-Hard problem.

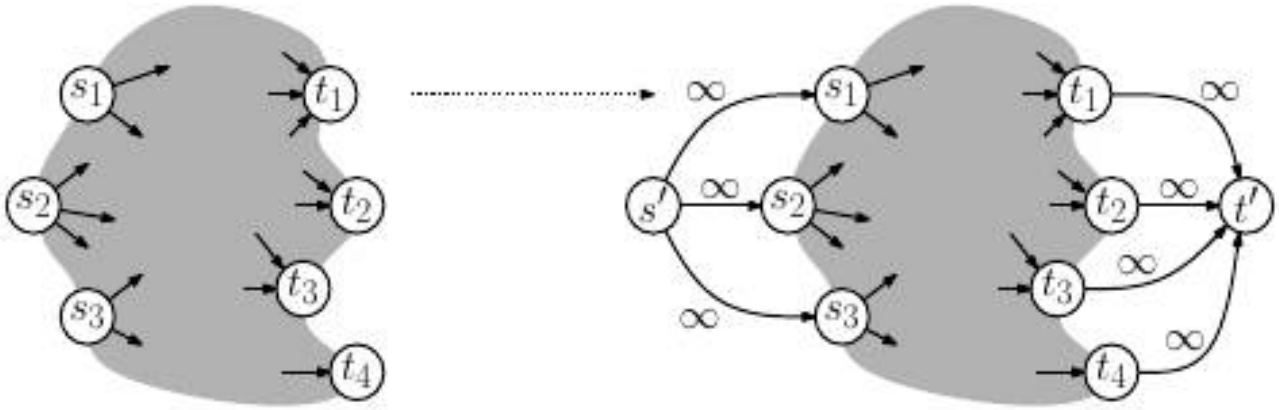


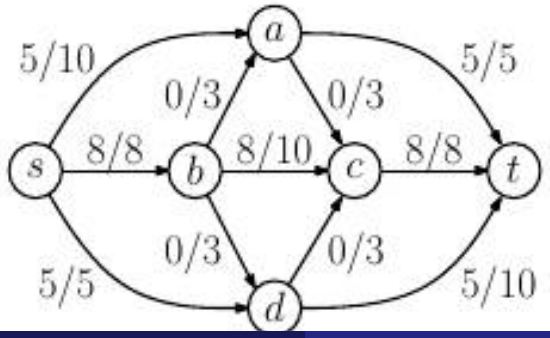
Table of Contents

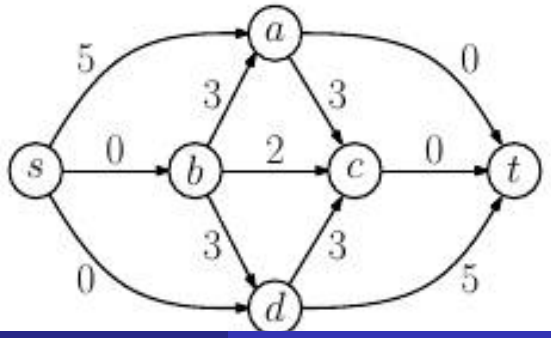
1. Preliminaries
2. Ford-Fulkerson Method
3. Analysis of Ford-Fulkerson Method
4. Maximum Flow (Max Flow) and Minimum Cut(Min Cut)
5. Maximum Bipartite Matching : A reduction to Maximum Flow Problem
6. Extension of Network Flows
   - Circulation
   - Circulation with upper and lower capacity bounds
   - Application: Survey design
7. Books

Path-based greedy algorithm works?

- Initially the flow on each edge is set to zero.
- Next, find any path P from s to t, such that the **edge capacities** on this path are all strictly positive. Let c_{min} be the **minimum capacity** of any edge on this path. This quantity is called the **bottleneck capacity** of the path.
- Push c_{min} units through this path. For each edge $(u, v) \in P$, set $f(u, v) = c_{min} + f(u, v)$, and **decrease** the capacity of (u, v) by c_{min}.
- Repeat this until no $s - t$ path (of positive capacity edges) remains in the network.

Note: It gives **positive flow** but **may not be the maximum flow**. **Left figure:** after pushing $5 + 5 + 8 = 18$ unit of flow,not the maximum flow. **Right figure:** No more $s - t$ path left after adjusting the capacities



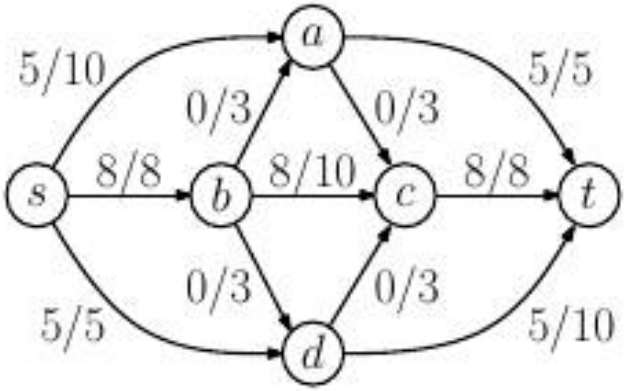


Problem of greedy algorithm and Residual network

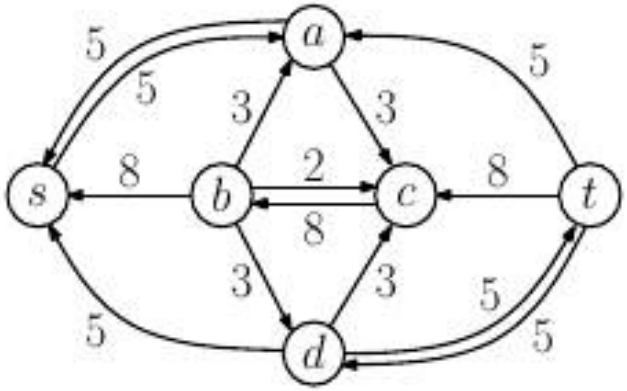
Need to adjust the flows through the $s - t$ path at later stage to push more flows in the network by **decreasing** the flow through already chosen paths; thus need to generate **Residual network**

- Given a flow network G and a flow f, **Residual network**, G_f, is a network having the same vertex set and same source and sink, and its **edges** are defined as follows:
  - **Forward edges:** For each edge (u, v) for which $f(u, v) < c(u, v)$, create an edge (u, v) in G_f with the capacity $c_f(u, v) = c(u, v) - f(u, v)$. Intuitively, in (u, v) we can still **push** $c_f(u, v)$ additional units of flow without violating the original capacity constraint.
  - **Backward edges:** For each edge (u, v) for which $f(u, v) > 0$, create an edge (v, u) in G_f with capacity $c_f(v, u) = f(u, v)$. Intuitively, we can **cancel** up to $f(u, v)$ units of flow along (u, v). Conceptually, by **pushing positive flow** along the **reverse edge** (v, u) we **decrease the flow** along the original edge (u, v).
- All edge capacities remains positive; for every edge in G, atmost two edges are created, thus space complexity remains as in original G

Residual Network: Illustration



(a): A flow f in network G



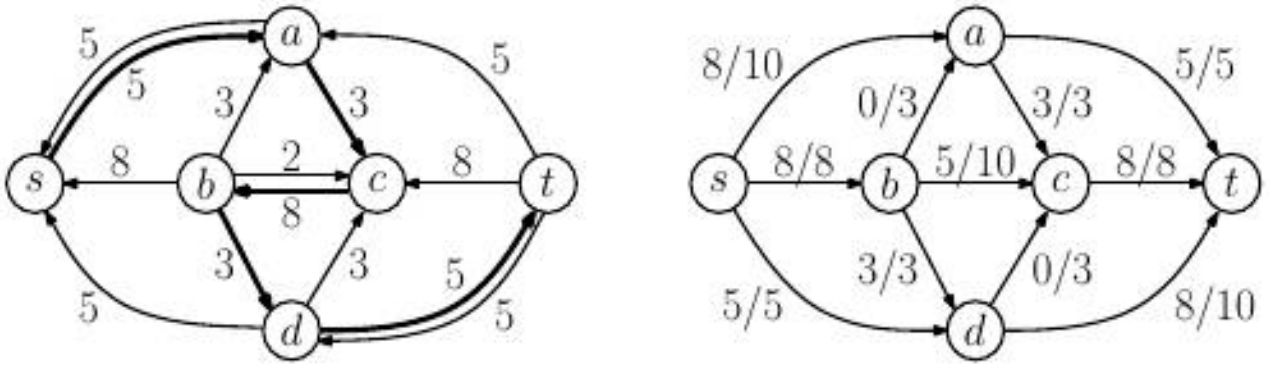
(b): Residual network G_f

- The capacity of each edge in the **residual network** G_f is called its **residual capacity**.
- Pushing flow through **residual network** is same as pushing additional amount of flow through original network.
- Given two flows f and f' , the sum of flows, $f + f'$ is sum of flows along each edge. If $f" = f + f'$, then $f''(u, v) = f(u, v) + f'(u, v)$. The value of $f + f'$ is equal to $|f| + |f'|$.
- **Intuition of algorithm:** To increase the flow is to find any flow in the residual network.

Augmenting Paths and Ford-Fulkerson (1)

Lemma: Let f be a flow in G and let f' be a flow in G_f. Then $(f + f')$ is a flow in G. (Prove later)

- Consider a network G, let f be a flow in G, and let G_f be the associated **residual network**. An **augmenting path** is a simple path P from s to t in G_f. The residual capacity, called **bottleneck capacity** of the path is the **minimum capacity of any edge on the path**, denoted by $c_f(P)$ ($c_f(P) > 0$).
- By pushing $c_f(P)$ units of flow along each edge of the path, we obtain a valid flow in G_f, and by the Lemma, adding this to f results in a valid flow in G of **strictly higher value**.



(a): Augmenting path of capacity 3 (b): The flow after augmentation

Augmenting Paths and Ford-Fulkerson (2)

- **Difference with greedy algo:** Greedy algo only **increases** flow on edges. But **augmenting path** may **increase flow** on a **backwards edge**, which may actually **decrease** the flow on some edge of the original network.
- **Ford-Fulkerson Method:** Start with a flow of 0, and then repeatedly **find an augmenting path** until no such path exists.

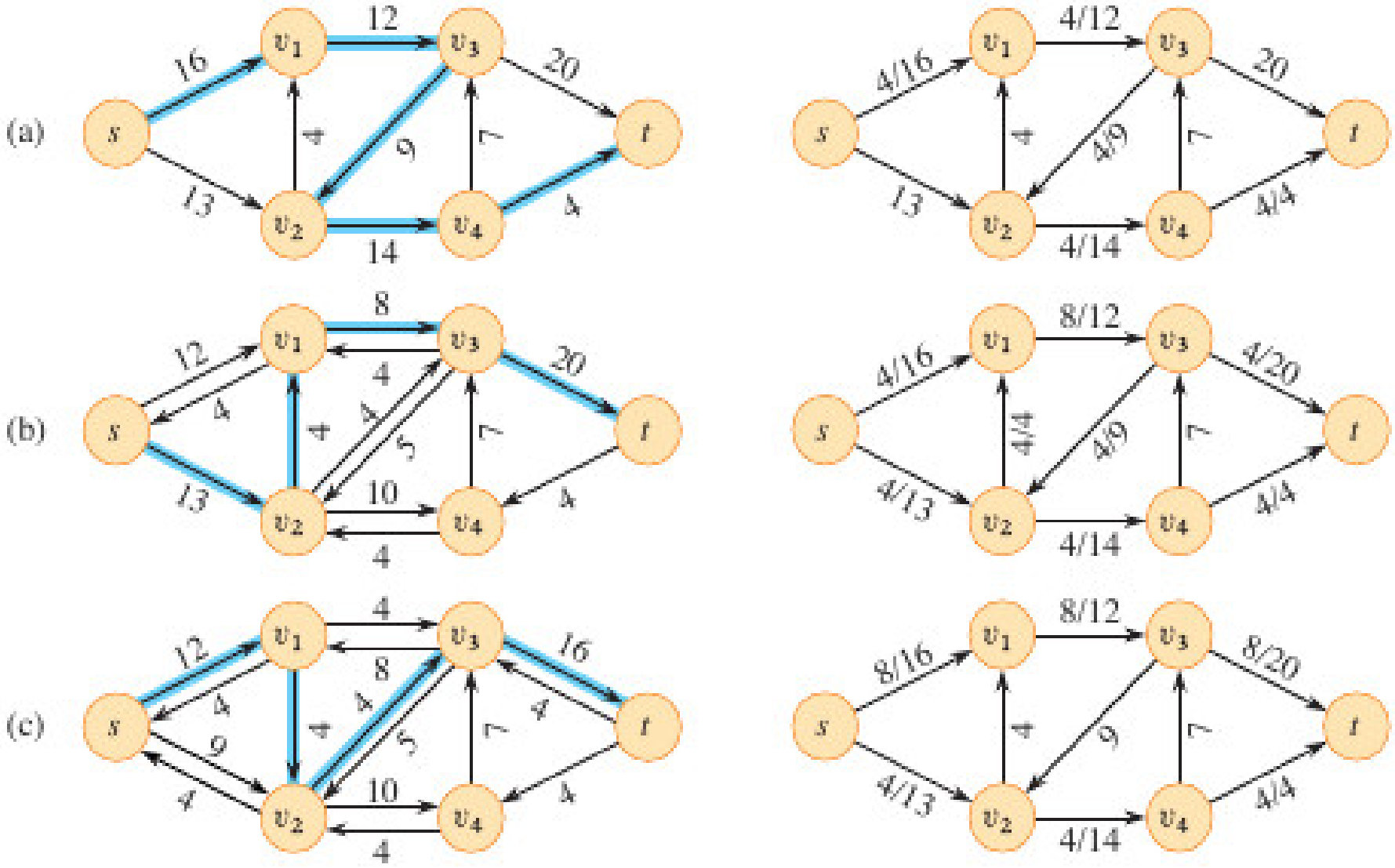
```
ford-fulkerson-flow(G = (V, E, s, t)) {
    f = 0 (all edges carry zero flow)
    while (true) {
        G' = the residual-network of G for f
        if (G' has no s-t augmenting path)
            break                                   // no augmentations left
        P = any-augmenting-path of G'               // augmenting path
        c = minimum capacity edge of P              // augmentation amount
        augment f by adding c to the flow on every edge of P
    }
    return f
}
```

Ford-Fulkerson Method: In Detail

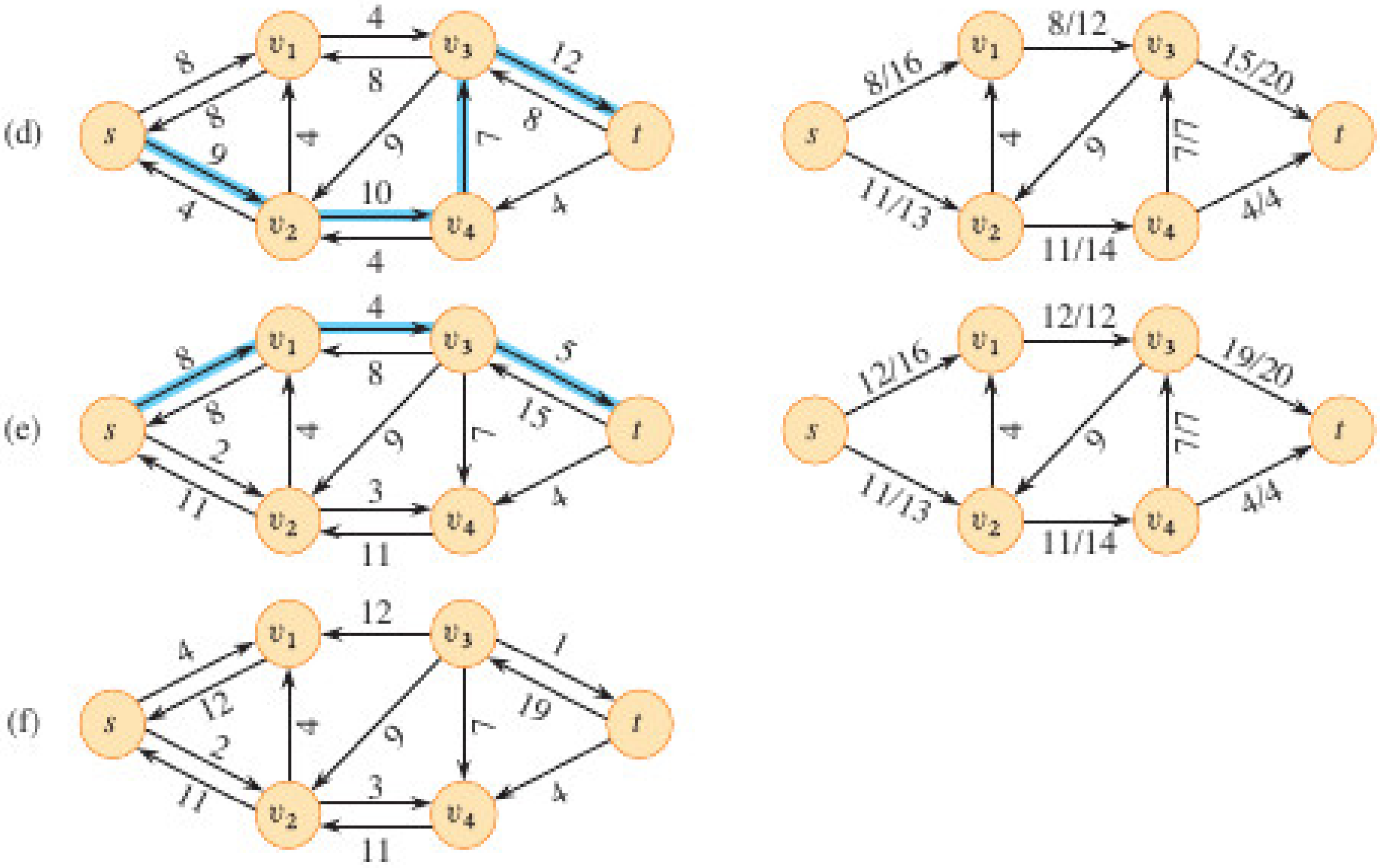
```
Augment(f,P)
Begin
  Let b = bottleneck(P,f) // b is minimum capacity among edges on path P
  For each edge (u,v) in P {
    If e = (u,v) is a forward edge then
      increase f(e) in G by b
    Else ((u,v) is a backward edge, and let e=(v,u))
      decrease f(e) in G by b
    Endif
  Endfor
  Return(f)
End

Max-Flow
Begin
  Initially f(e) = 0 for all e=(u,v) in G
  While there is an s-t path in the residual graph Gf
    Let P be a simple s-t path in Gf
    f' = augment(f,P)
    Update f to be f'
    Update the residual graph Gf to be Gf'
  Endwhile
  Return f
End
```

Ford-Fulkerson Method: Illustration (1)



Ford-Fulkerson Method: Illustration (2)



Ford-Fulkerson Method: Issues

1. How efficiently can we perform augmentation?
2. How many augmentations might be required until converging?
3. If no more augmentations can be performed, have we found the max-flow?

1. How efficiently can we perform augmentation?

- Given G and f, creating G_f takes $O(n+m)$ time, where $n = |V|$ and $m = |E|$.
- Run BFS or DFS to find an augmenting path from s to t in G_f, if one exists. Let P be the resulting path. BFS or DFS runs in $O(n+m)$ time.
- Finally, compute the minimum capacity edge along P, increase the flow f by this amount for every edge of P, that takes $O(m)$ time.
- **But** how many times while loop runs? **The second question**

Table of Contents

1. Preliminaries
2. Ford-Fulkerson Method
3. Analysis of Ford-Fulkerson Method
4. Maximum Flow (Max Flow) and Minimum Cut(Min Cut)
5. Maximum Bipartite Matching : A reduction to Maximum Flow Problem
6. Extension of Network Flows
   - Circulation
   - Circulation with upper and lower capacity bounds
   - Application: Survey design
7. Books

Lemma 1(1)

Lemma 1: f' is a flow in G, where f' is obtained after augmenting G with bottleneck capcity b to the flow on every edge of P
Proof: Verify capacity constraint and flow conservation condition

- f' differs from f only on edges of P, check capacity constraint and flow conservation condition on those edges (u, v) on P;
- If (u, v) is **forward edge**,flow increases only till $c(u, v)$; its **residual capacity** is $c(u, v) - f(u, v)$. If (v, u) is corresponding **backward edge**, flow can not be reduced below 0; b the bottleneck capcity (c in Pseudocode) is **no larger than** residual capacity of (u, v);
- For fwd edge (u, v): $0 \leq f(u, v) \leq f'(u, v)$, where $f'(u, v) = f(u, v) + b \leq f(u, v) + c(u, v) - f(u, v) = c(u, v)$; thus, $f'(u, v) \leq c(u, v)$, **capacity constraint holds**
- For bwd edge (v, u): the residual capcity is $f(v, u)$; $c(v, u) \geq f(v, u) \geq f'(v, u)$, where $f'(v, u) = f(v, u) - b \geq f(v, u) - f(v, u) = 0$; thus, $c(v, u) \geq f(v, u) \geq 0$, **capacity constraint holds** (never becomes negative)

Lemma 1 and Lemma 2(2)

- Let v be a node on path P; Verify that **change** in flow entering v and **change** in flow exiting v is same.
- Since, f satisfied flow conservation at v, so must f'
- In other words, sum of flows in G and G_f is a flow in G

Lemma 2: At every intermediate stage of the Ford-Fulkerson Algorithm, the flow values $\{f(u,v)\}$ and the residual capacities in G_f are integers.
Proof:

- The statement is true before any iterations of the While loop. Suppose it is true after j iterations.
- Since all residual capacities in G_f are **integers**, the value $bottleneck(P, f)$ for the augmenting path found in iteration $j+1$ will be an integer.
- Thus the flow f' will have **integer values**, and hence the capacities of the new residual graph G'_f

Lemma 3 (3)

Lemma 3: Let f be a flow in G, and let P be a simple $s-t$ path in G_f. Then $|f'| = |f| + bottleneck(P, f)$; and since $bottleneck(P, f) > 0$, we have $|f'| > |f|$.

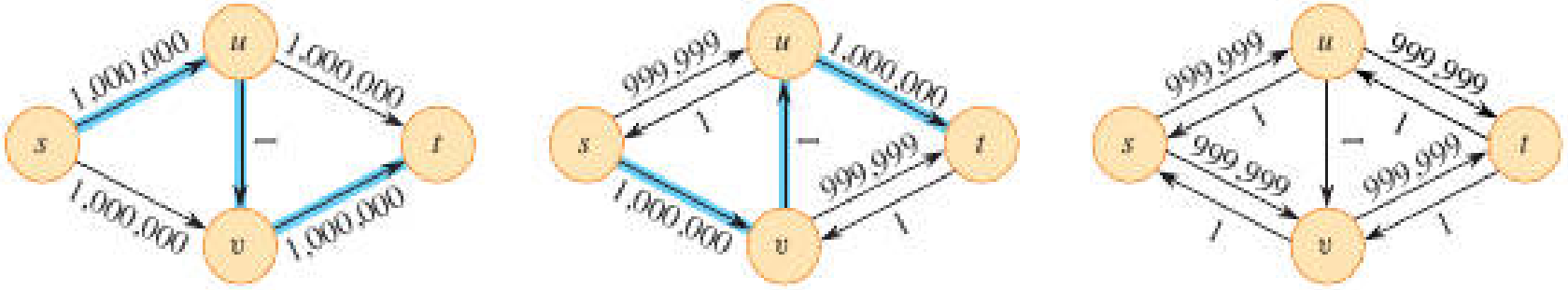
Proof:

- The first edge e of P must be an edge out of s in the residual graph G_f ; and since the path is simple, it does not visit s again.
- Since G has no edges entering s, the edge e must be a **forward edge**. We increase the flow on this edge by $bottleneck(P, f)$, and we do not change the flow on any other edge incident to s. Therefore the value of f' exceeds the value of f by $bottleneck(P, f)$.

Lemma 4 (4)

Lemma 4: Let $C = \sum_{\text{e out of s}} c(e)$ and all $c(e)$ are integers. Then Ford-Fulkerson Method terminates in at most C iterations of While loop. (Answers Question 1 and 2) **Proof:**

- No flow in G can have value greater than C, due to the capacity condition on the edges leaving s.
- Now, by Lemma 3, the value of the flow maintained by the Ford-Fulkerson method increases in each iteration;
- By Lemma 2, it increases by at least 1 in each iteration.
- Since it starts with the value 0, and cannot go higher than C, the While loop in the Ford-Fulkerson method can run for at most C iterations.



Lemma 5 (5)

Lemma 5: The Ford-Fulkerson method can be implemented to run in $O(mC)$ time.

- From Lemma 4, method terminates in at most C iterations of While loop.
- Consider the amount of work involved in one iteration when the current flow is f.
- The residual graph G_f has at most $2m$ edges, since each edge of G gives rise to at most two edges in the residual graph.
- Consider G_f be implemented as adjacency list representation having two linked lists for each node v, for incoming and outgoing edges of v.
- To find an $s - t$ path in G_f , we can use breadth-first search or depth-first search, which run in $O(m + n)$ time or $O(m)$ time;
- $Augment(f, P)$ takes time $O(n)$, as path P has at most $n - 1$ edges.
- Given the new flow f', we can build the new residual graph in $O(m)$ time: For each edge e of G, we construct the correct forward and backward edges in G'_f.
- Thus, time taken is $O(mC)$

Table of Contents

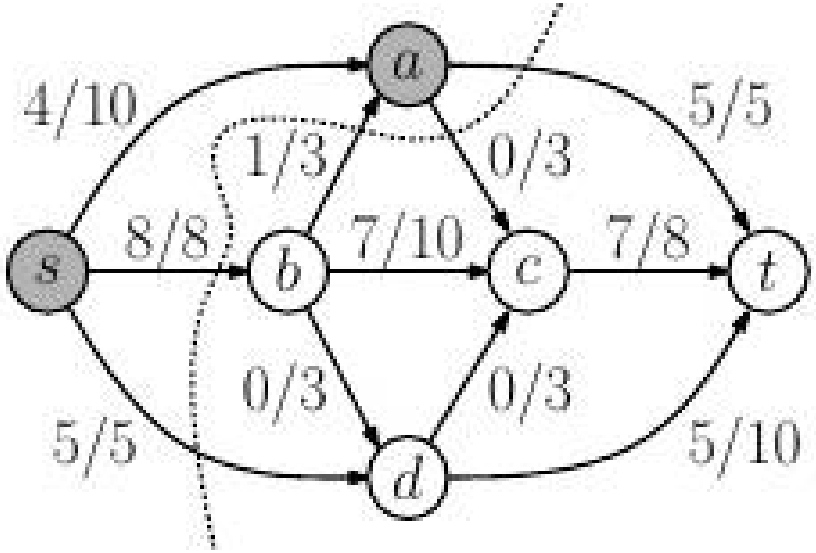
1. Preliminaries
2. Ford-Fulkerson Method
3. Analysis of Ford-Fulkerson Method
4. Maximum Flow (Max Flow) and Minimum Cut(Min Cut)
5. Maximum Bipartite Matching : A reduction to Maximum Flow Problem
6. Extension of Network Flows
   - Circulation
   - Circulation with upper and lower capacity bounds
   - Application: Survey design
7. Books

Why a Cut?

- Our next goal is to show that the flow that is returned by the Ford-Fulkerson Algorithm has the **maximum possible value of any flow** in G.
- Observe the way the structure of the flow network places **upper bounds** on the maximum value of an $s - t$ flow; $C = \sum_{e \text{ out of } s} c(e)$
- The flow cannot be increased forever, because there is some subset of edges, whose capacities eventually become **saturated with flow** and there is no more $s - t$ path.
- Every path from s to t **must cross one of these saturated edges**; thus, the **sum of capacities** of these edges imposes an **upper bound** on value of **maximum flow** and these edges form a bottleneck.
- Since such a set of **saturated edges** lie on every path from s to t, their removal defines a **partition** (**A Cut**) separating the vertices that s can reach and the set of vertices that s cannot reach.

Cut

- Given a network G, define a **cut** (also called an $s-t$ cut) to be a partition of the vertex set into two **disjoint subsets** $X \subseteq V$ and $Y = V \backslash X$, where $s \in X$ and $t \in Y$.
- We define the **net flow** from X to Y to be the **sum of flows from** X **to** Y **minus the sum of flows from** Y **to** X .
  $f(X,Y) = \sum_{x \in X} \sum_{y \in Y} f(x,y) - \sum_{y \in Y} \sum_{x \in X} f(y,x)$
- Note that $f(X,Y) = -f(Y,X)$



$X = \{s, a\}$

$Y = \{b, c, d, t\}$

$f(X,Y) = 5 + 0 - 1 + 8 + 5 = 17$

Lemma 6

Lemma 6: Let (X, Y) be any $s - t$ cut in a network. Given any flow f , the value of f is equal to the net flow across the cut, that is, $f(X, Y) = |f|$.
Proof:

- Note that there are no incoming edges to s, and so we have $|f| = f_{out}(s) = f_{out}(s) - f_{in}(s)$.
- Since all the other nodes of X must satisfy flow conservation it follows that $|f| = \sum_{x \in X} (f_{out}(x) - f_{in}(x))$
- Note that every edge (u, v) where both u and v are in X contributes one positive term and one negative term of value $f(u, v)$ to the above sum, and so all of these cancel out. The only terms that remain are the edges that either go from X to Y (which contribute positively) and those from Y to X (which contribute negatively).
- Thus, it follows that the value of the sum is exactly $f(X, Y)$, and therefore $|f| = f(X, Y)$.

Capacity of a cut

- **Capacity of the cut** (X, Y) is the sum of the capacities of the edges leading from X to Y , that is, $c(X, Y) = \sum_{x \in X} \sum_{y \in Y} c(x, y)$
- Note that it is not possible to push more flow through a cut than its capacity.

Lemma 7: Given any $s - t$ cut (X, Y) and any flow f we have $|f| \leq c(X, Y)$.
Proof:

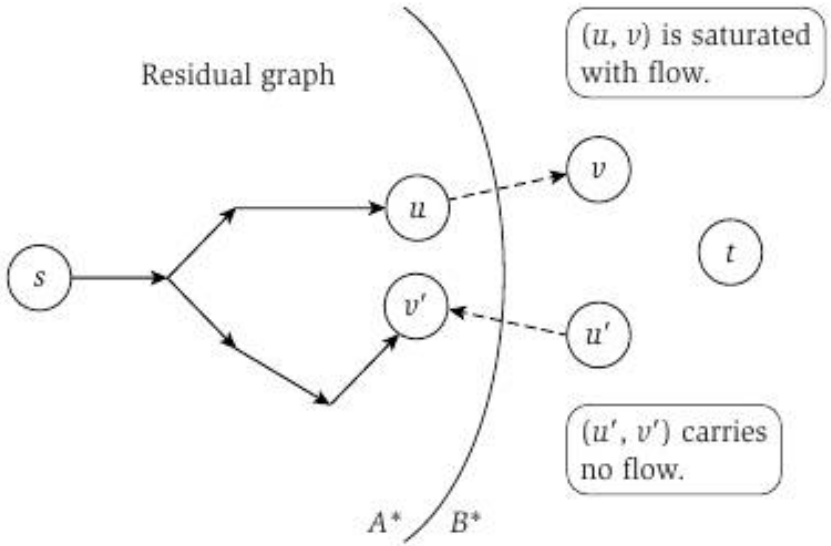
$$\begin{aligned} |f| &= f_{out}(X) - f_{in}(X) \\ &\leq f_{out}(X) \\ &= \sum_{e \text{ out of } X} f(e) \\ &\leq \sum_{e \text{ out of } X} c(e) \\ &= c(X, Y) \end{aligned}$$

Max-flow Min-cut Theorem (1)

If f is an $s-t$ flow such that there is no $s-t$ path in the residual graph G_f, then there is an $s-t$ cut $(A*, B*)$ in G for which $|f| = c(A*, B*)$. Consequently, f has the **maximum value of any flow** in G, and $(A*, B*)$ has the **minimum capacity** of any $s-t$ cut in G.

Proof:

- Let $A*$ denote the set of all nodes v in G for which there is an $s-v$ path in G_f. Let $B*$ denote the set of all other nodes: $B* = V - A*$.
- Establish that $(A*, B*)$ is indeed an $s-t$ cut. The source s belongs to $A*$ since there is always a path from s to s. Moreover, $t \notin A*$ as there is no $s-t$ path in the residual graph; hence $t \in B*$ as desired.



Max-flow Min-cut Theorem (2)

- Let $e = (u, v)$ is an edge in G for which $u \in A*$ and $v \in B*$. We claim that $f(e) = c(e)$. For if not, e would be a **forward edge** in G_f , and since $u \in A*$, there is an $s - u$ path in G_f ; appending e to this path, will yield an $s - v$ path in G_f, contradicting the assumption that $v \in B*$.
- Let $e' = (u', v')$ is an edge in G for which $u' \in B*$ and $v' \in A*$. We claim that $f(e') = 0$. For if not, e' would give rise to a **backward edge** e" $= (v', u')$ in G_f , and since $v' \in A*$, there is an $s - v'$ path in G_f ; appending e'' to this path, would result an $s - u'$ path in G_f, contradicting the assumption that $u' \in B*$.
- So all edges out of $A*$ are completely **saturated** with flow, while all edges into $A*$ are completely **unused**. Using the fact that $|f| = f_{out}(A) - f_{in}(A)$:

$$
\begin{aligned}
|f| &= f_{out}(A*) - f_{in}(A*) \\
&= \sum_{e \text{ out of } A*} f(e) - \sum_{e \text{ into } A*} f(e) \\
&= \sum_{e \text{ out of } A*} f(e) - 0 \\
&= c(A*, B*).
\end{aligned}
$$

Max-flow Min-cut Theorem (3)

Given that the Ford-Fulkerson Method terminates when there is no $s - t$ in the residual graph, by Max-flow Min-cut Theorem the flow is maximum.

Alternatively: The following three conditions are equivalent.

(i) f is a maximum flow in G,

(ii) The residual network G_f contains no augmenting paths,

(iii) $|f| = c(X, Y)$ for some cut (X, Y) of G.

Proof:

- $(i) \implies (ii)$: (by contradiction) If f is a max flow and there were an augmenting path in G_f , then by pushing flow along this path we would have a larger flow, a contradiction.
- $(ii) \implies (iii)$: If there are no augmenting paths then s and t are not connected in the G_f. Let X be those vertices reachable from s and Y be the rest in G_f. (X, Y) forms a cut. Because each edge crossing the cut must be saturated with flow, it follows $|f| = c(X, Y)$.
- $(iii) \implies (i)$: (by contradiction) Suppose that there is a flow f' whose value exceeds $|f|$. Then we would have $|f'| > c(X, Y)$, which contradicts the previous lemma $|f| \leq c(X, Y)$

Max-flow Min-cut Theorem (4)

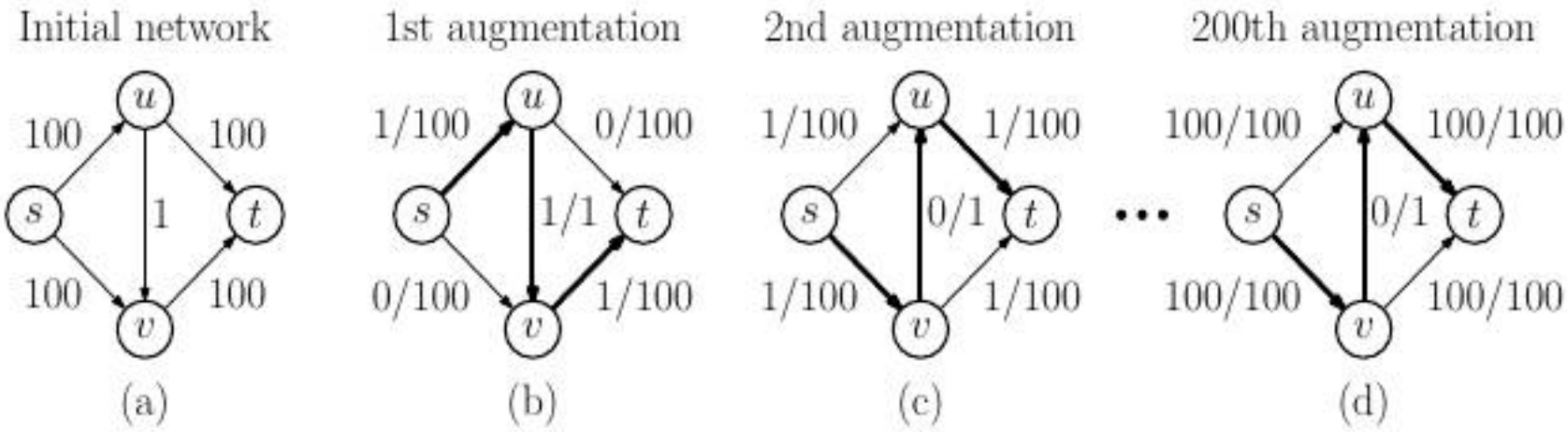
Lemma: Given a flow f of maximum value, an $s - t$ cut of minimum capacity is computed in $O(m)$ time.

Proof:

Construct the residual graph G_f, and perform breadth-first search or depth-first search to determine the set $A*$ of all nodes that s can reach. We then define $B* = V - A*$, and return the cut $(A*, B*)$

Ford-Fulkersom Method: Issues

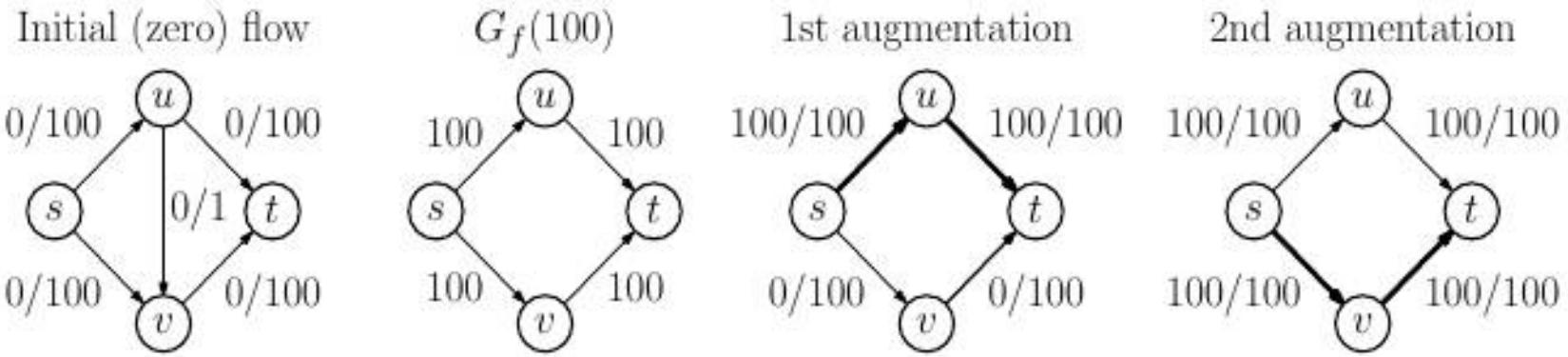
How to Choose Augmenting Path?



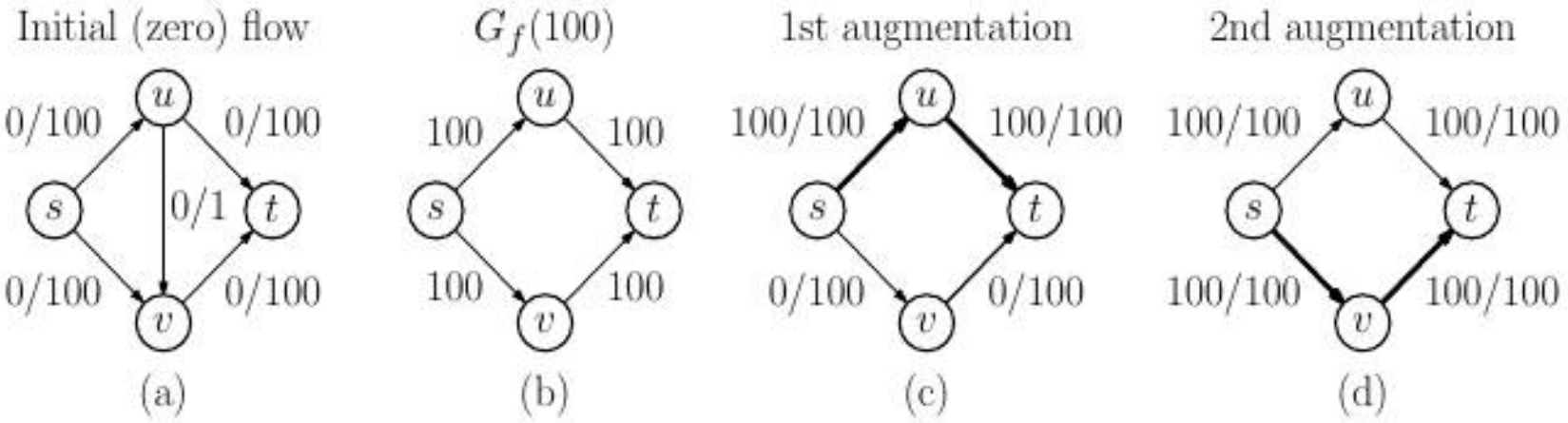
- If we let $F*$ denote the final maximum flow value, the number of augmentation steps can be as high as $F*$. If each augmentation step takes at least $\Omega(m)$ time, the total running time can be as high as $\Omega(F * \cdot m)$. Since $F*$ may be arbitrarily high (it depends neither on n or m) making running time arbitrarily high.
- **Scaling FF:** Push higher flow values in the beginning and if no such flow is found, half the amount and try finding augmenting path for it.

Scaling Ford-Fulkersom Method

- Let $C = \sum_{(s,u)\in E} c(s,u)$; start pushing flow with upper bound of flow
- Define $\Delta \leq C$, where Δ is power of 2
- Given any flow f (initially 0), define $G_f(\Delta)$ to be the residual network consisting only of edges of residual capacity at least Δ.
- Repeatedly find an augmenting path in $G_f(\Delta)$, augment the flow along this path, and then compute the $G'_f(\Delta)$ for the augmented flow f' until no augmenting paths remain.
- When no more augmenting paths remain, set $\Delta \leftarrow \Delta/2$, compute $G_f(\Delta)$ for the new value of Δ, and repeat the process.
- Eventually, $\Delta = 1$, algorithm terminates and yields the final maximum flow.



Scaling Ford-Fulkersom Method



- Initially $\Delta = 200$; $G_f(200)$ is empty as no edge with capcity $\geq \Delta$ (Figure (a))
- $G_f(100)$ includes the edges of capcity only 100 and not 1 (Figure (b))
- After two augmentations: (Figure (c) and (d))
- Then it tries $G_f(50), G_f(25) \cdots G_f(1)$. But we have already found Max flow.

Scaling Ford-Fulkersom Method: Time complexity (1)

- For each choice of Δ, the algorithm terminates after $O(m)$ augmentation steps.
  Proof:
  - The statement is true in the first scaling phase, as we can use each of the edges out of s only for at most one augmentation in that phase.
  - In later scaling phase Δ, let f_p be the flow at the end of the previous scaling phase.
  - In previous phase, $\Delta' = 2\Delta$. Maximum flow $f*$ is at most $|f*| \leq |f_p| + m\Delta' = |f_p| + 2m\Delta$.
  - In Δ-scaling phase, each augmentation increases the flow by at least Δ, and thus there can be at most $2m$ augmentations.
- Thus, the time spent for each value of Δ is $O(m^2)$.

Scaling Ford-Fulkersom Method: Time complexity (2)

- Finally, since the value of Δ is halved with each iteration, $O(\log C)$ different values of Δ is required.
- Whenever C is sufficiently large (when $C/\log C$ is asymptotically larger than m) the scaling algorithm will outperform the Ford-Fulkerson algorithm.
- Scaling algo. is polynomial in the number of bits needed to encode the input. Thus,it is said to run in pseudo-polynomial time.
- Time complexity: $O(m^2 \log C)$

Other improved Algorithms exists: Edmond Karp ($O(nm^2)$), Preflow-Push ($O(n^3)$), Tarjan's Algo ($O(nm)$)

Table of Contents

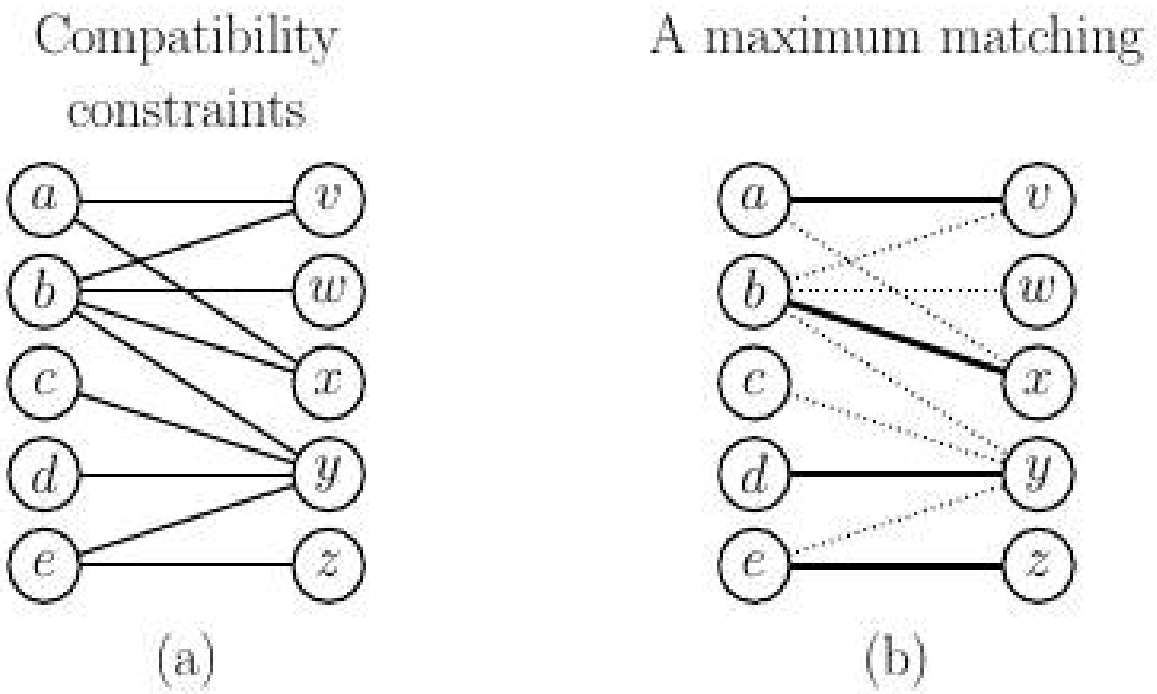
1. Preliminaries
2. Ford-Fulkerson Method
3. Analysis of Ford-Fulkerson Method
4. Maximum Flow (Max Flow) and Minimum Cut(Min Cut)
5. Maximum Bipartite Matching : A reduction to Maximum Flow Problem
6. Extension of Network Flows
   - Circulation
   - Circulation with upper and lower capacity bounds
   - Application: Survey design
7. Books

Match Making Problem

Match Making Problem of Matrimonial site: Given the preferences of W women and M men, find maximum possible matching pair of women and men

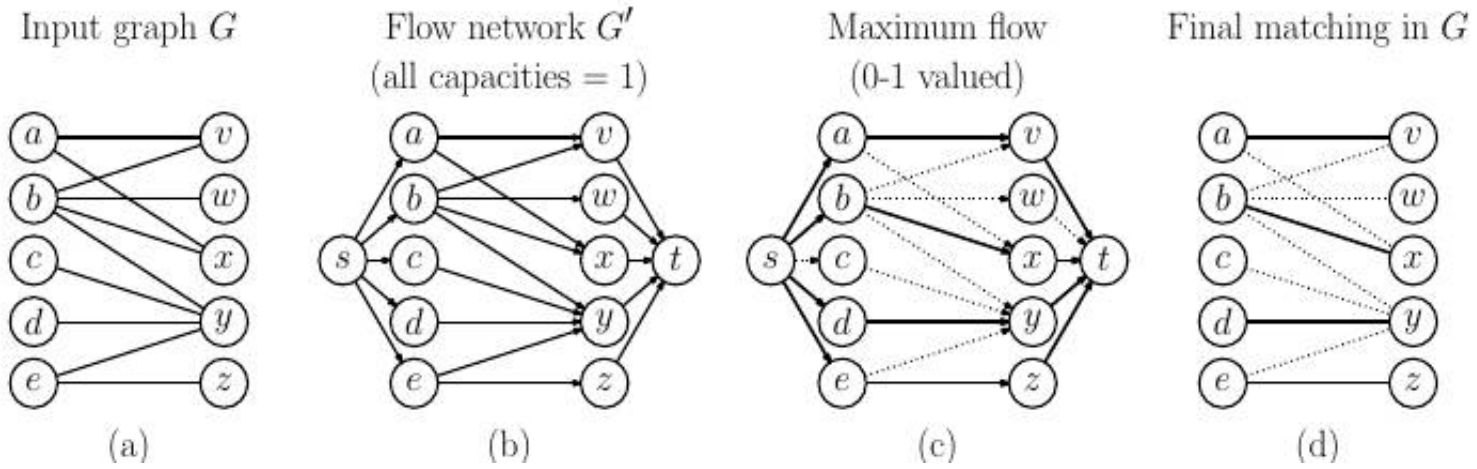
- **Modeling the problem:** An undirected graph $G = (V, E)$, is named **bipartite graph** if V is partitioned into two sets X and Y, such that every edge $e \in E$ has one endpoint in X and the other in Y. Thus, the model is: $V = X \cup Y$ and edge set consists of pairs $(u, v), u \in X, v \in Y$ such that u and v are compatible.
- Given a graph, a **matching** is defined to be a subset of edges $M \subseteq E$ such that for each $v \in V$, there is at most one edge of M incident to v. Thus, for the **match making problem, find a maximum matching in G that has the highest cardinality**. Such a matching is called a **maximum matching**
- The **match making** problem is **Maximum Bipartite Matching Problem**, as match making problem is modeled as a **bipartite graph**.

Maximum Bipartite Matching(MBM): Reduction to Maximum Flow Problem (1)



- Given a **bipartite graph** G to find maximum matching, convert G to an instance of network flow G', such that the **maximum matching** on G can be extracted from the **maximum flow** on G' .

MBM: Reduction to Maximum Flow Problem (2)

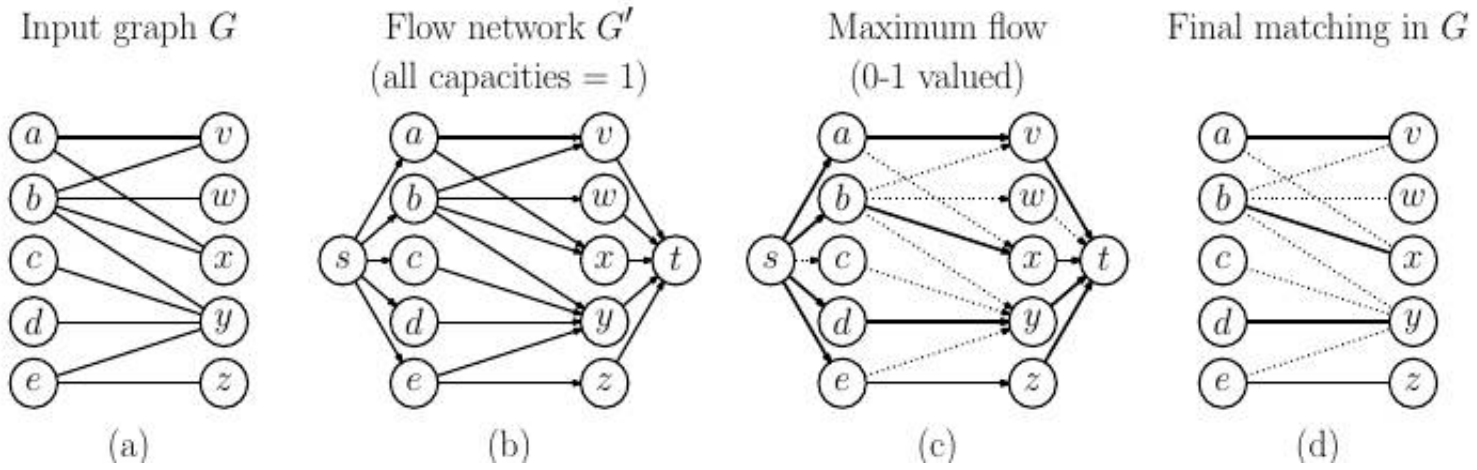


- Construct a flow network $G' = (V', E')$ as follows. Let s and t be two new vertices and let $V' = V \cup s, t$.

$$E' = \begin{cases} \{(s,u) | u \in X\} \cup & \text{(connect source to left-side vertices)} \\ \{(v,t) | v \in Y\} \cup & \text{(connect right-side vertices to sink))} \\ \{(u,v) | (u,v) \in E\} & \text{direct } G\text{'s edges from left to right)} \end{cases}$$

- Set the capacity of all edges in this network to 1
- Compute the maximum flow in G'.
- Since each vertex in X has exactly **one incoming edge**, it can have flow along at most one outgoing edge, and since each vertex in Y has exactly **one outgoing edge**, it can have flow along at most one incoming edge.

MBM: Reduction to Maximum Flow Problem (2)



- Thus for f, the maximum flow, we can define a matching $M = \{(u, v) | u \in X, v \in Y, f(u, v) > 0\}$
- This matching is **maximum** because for **every matching there is a corresponding flow of equal value**, and for every (integer) flow there is a matching of equal value.
- The capacities are low and thus $F = |f|$ is not very large. The total capacity of the edges coming out of s is at most $|X| \leq |V| = n$.
- Therefore, the running time of Ford-Fulkerson on this instance is $O(m \cdot F) = O(nm)$.

Better Algorithm exists: Hopcroft and Karp that runs in $O(\sqrt{n}m)$ time.

Table of Contents

1. Preliminaries
2. Ford-Fulkerson Method
3. Analysis of Ford-Fulkerson Method
4. Maximum Flow (Max Flow) and Minimum Cut(Min Cut)
5. Maximum Bipartite Matching : A reduction to Maximum Flow Problem
6. Extension of Network Flows
   - Circulation
   - Circulation with upper and lower capacity bounds
   - Application: Survey design
7. Books

Table of Contents

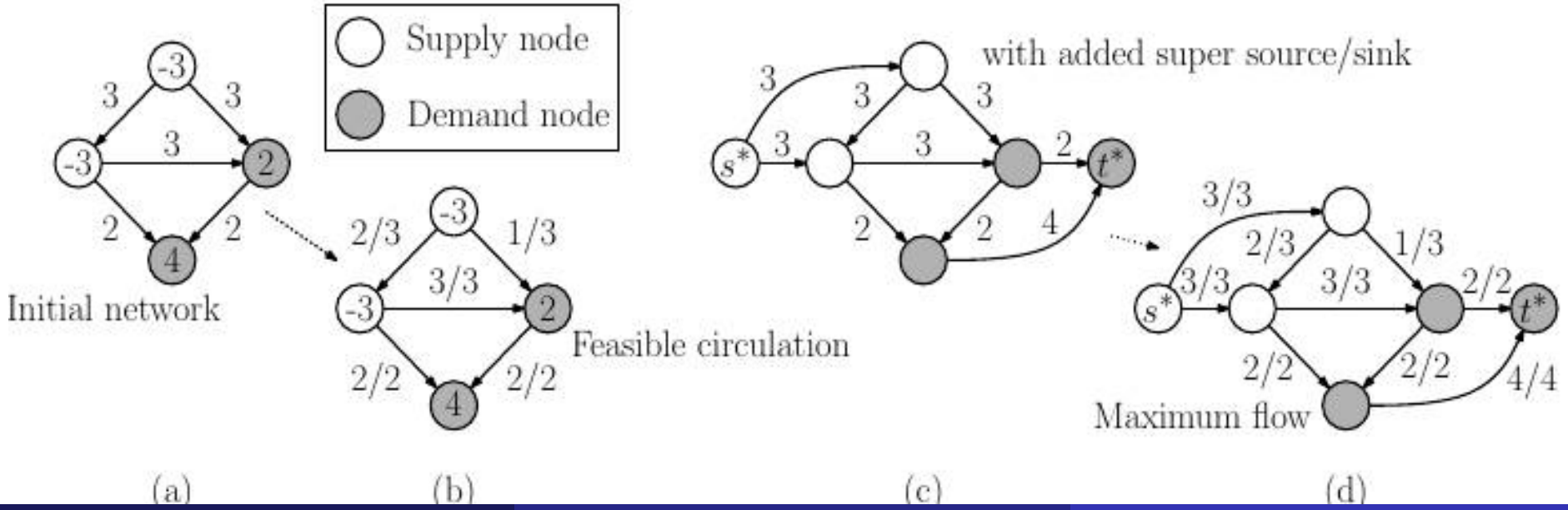
1. Preliminaries
2. Ford-Fulkerson Method
3. Analysis of Ford-Fulkerson Method
4. Maximum Flow (Max Flow) and Minimum Cut(Min Cut)
5. Maximum Bipartite Matching : A reduction to Maximum Flow Problem
6. Extension of Network Flows
   - Circulation
   - Circulation with upper and lower capacity bounds
   - Application: Survey design
7. Books

Circulation with Demands

- A collection of **supply nodes** want to ship flow (or products or goods) and a collection of **demand nodes** want to receive flow.
- Each **supply node** is associated with the amount of product it wishes to **ship** and each **demand node** is associated with the amount that it wishes to **receive**.
- **The Problem:** Find if there is some way to get the products from the supply nodes to the demand nodes, subject to the **capacity constraints**. This is a decision problem (or feasibility problem) with yes-no answer, as opposed to maximum flow, which is an optimization problem.
- **How to model demand?** A single numeric value is associated with each node, called its **demand**. If $v \in V$ is a **demand node**, then d_v is the amount of this demand. If v is a **supply node**, then $-d_v$ is associated with v

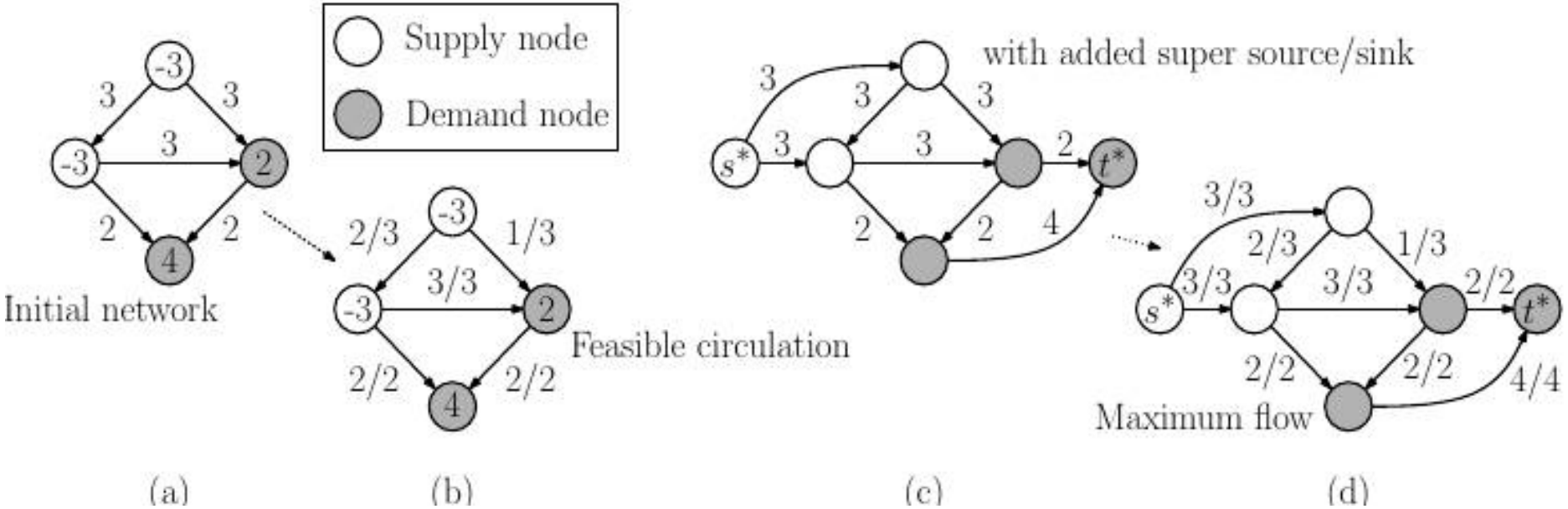
Circulation with Demands: Problem Statement

- Given a directed graph $G = (V, E)$ in which each edge (u, v) is associated with a positive capacity $c(u, v)$ and each vertex v is associated with a **supply/demand value** d_v. Let S denote the set of supply nodes ($d_v < 0$), and let T denote the set of demand nodes ($d_v > 0$). Note that vertices of S may have incoming edges and vertices of T may have outgoing edges. (Figure (a))
- A **circulation** in G is a function f that assigns a non-negative real number to each edge that satisfies the following two conditions.
  - **Capacity constraints:** For each $(u, v) \in E$, $0 \leq f(u, v) \leq c(u, v)$.
  - **Supply/Demand constraints:** For vertex $v \in V$, $f_{in}(v) - f_{out}(v) = d_v$.



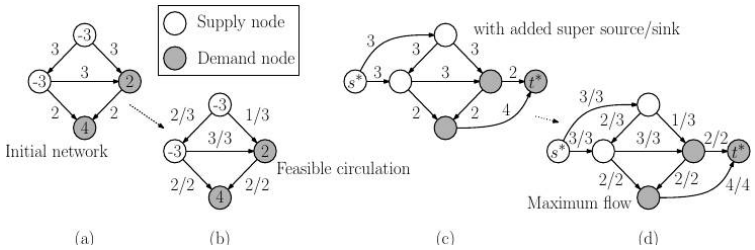
Circulation with Demands: Observations

- **Demand constraints** correspond to the flow-balance in the original max flow problem, since if a vertex v is not in S or T , then $d_v = 0$ and $f_{in}(v) = f_{out}(v)$.
- **Total demand** must equal the **total supply**, otherwise no chance of finding a feasible circulation. That is, we require that $\sum_{v \in V} d_v = 0$ or equivalently $-\sum_{v \in S} d_v = \sum_{v \in T} d_v$
- **Total demand:** $D = \sum_{v \in T} d_v, \forall v \in T$ This is equal to **Total supply** $\sum_{v \in S} d_v$



Circulation: Reduction to Network Flow Problem

- Convert any instance $G = (V, E)$ with vertex demands d_v and edge capacities $c(u, v)$ of the circulation problem to an **equivalent network flow problem**. Assume that total supply equals total demand, otherwise there is no circulation. (Figure (c))
- Create a new network $G' = (V', E')$ that has all the same vertices and edges as G (that is, $V' = V$ and $E' = E$)
- Add to V' a super-source $s*$ and a super-sink $t*$
- For each **supply node** $v \in S$, we add a new edge $(s*, v)$ of capacity $-d_v$
- For each **demand node** $u \in T$, we add a new edge $(u, t*)$ of capacity d_v
- On **running FF** on G', if the **flow value is at least** D, then intuitively, there is enogh flow into the network and (by flow balance) enough flow out of the network to **satisfy all the demand constraints** (Figure (d)).
- **Time complexity:** Creating G' takes $O(m + n)$(reduction time); Time complexity of network flow algorithm



Circulation : Lemma

Lemma: There is a feasible circulation in G if and only if G' has an $s* - t*$ flow of value D.

- **Proof:** Suppose f is a feasible circulation in G. The value of this circulation (the net flow coming out of all supply nodes) is D.
- We can create a flow f' of value D in G', by saturating all the edges coming out of $s*$ and all the edges coming into $t*$. We claim that this is a valid flow for G'; it satisfies all capacity constraints.
- It satisfies the **flow balance constraints**. For each vertex $v \in V$:
  - $(v \in S)$: The flow into v from $s*$ matches the supply coming out of v from the circulation.
  - $(v \in T)$: The flow out of v to $t*$ matches the demand coming into v from the circulation.
  - $(v \in V \backslash (S \cup T))$: $d_v = 0$, so it already satisfied flow constraint.
- **Conversely**, let $|f'|$ is D in G'. Then it each edge leaving $s*$ and each edge entering $t*$ is saturated. Thus, by flow balance of f' , all the supply and demand nodes have achieved their desired supply/demand quotas. Ignoring the flows along the edges incident to $s*$ and $t*$, we have a feasible circulation f for G.

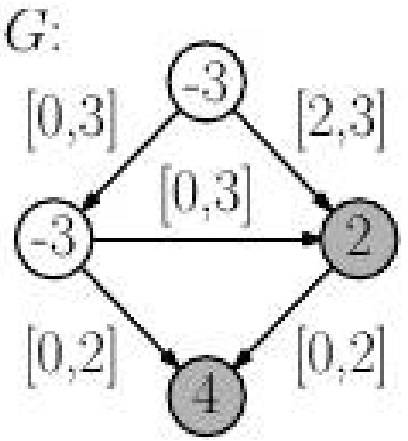
Table of Contents

1. Preliminaries
2. Ford-Fulkerson Method
3. Analysis of Ford-Fulkerson Method
4. Maximum Flow (Max Flow) and Minimum Cut(Min Cut)
5. Maximum Bipartite Matching : A reduction to Maximum Flow Problem
6. Extension of Network Flows
   - Circulation
   - Circulation with upper and lower capacity bounds
   - Application: Survey design
7. Books

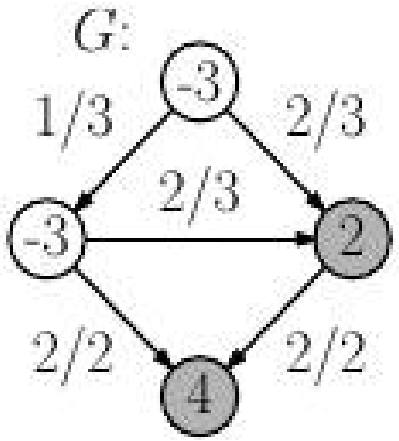
Circulation with Upper and Lower Capacity Bounds

Problem: Given a network $G = (V, E)$, for each edge $(u, v) \in E$ we would like to specify two constraints $l(u, v)$ and $c(u, v)$, where $0 \leq l(u, v) \leq c(u, v)$. A circulation function f must satisfy the same demand constraints as before, but must also satisfy both the **upper and lower flow bounds**:

- **(New) Capacity Constraints**: For each $(u, v) \in E, l(u, v) \leq f(u, v) \leq c(u, v)$.
- **Demand Constraints**: For vertex $v \in V, f_{in}(v) - f_out(v) = d_v$.



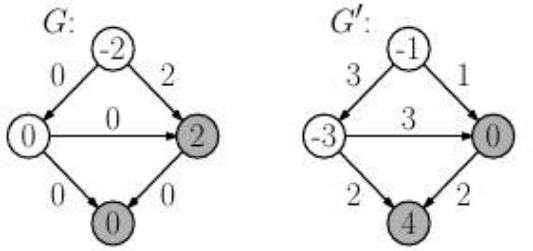
Initial network
(edges labeled with $[\ell, c]$ bounds)



A valid flow

Circulation with Capacity Bounds: Intuition (1)

- Suppose we generate an initial, invalid circulation f^0. It must satisfy lower flow bounds but may not satisfy demand constraint; $f^0(u,v) = l(u,v)$
- Modify the supply/demand values to **compensate for this imbalance**. Since the lower-bound constraints are all satisfied, apply a standard circulation algorithm (without lower flow bounds) to solve the problem.
- For each $v \in V$, let L_v denote the **excess flow coming into** v in f^0, that is $L_v = f^0_{in}(v) - f^0_{out}(v) = \sum_{(u,v)\in E} l(u,v) - \sum_{(v,w)\in E} l(v,w)$ L_v may be negative, which means a **flow deficit**.
- If we are lucky, $L_v = d_v$, and v's supply/demand needs are already met.
- Otherwise, adjust the supply and demand values so that by
  - computing any valid circulation f^1 for the adjusted values and
  - combining this with f^0. we obtain a flow satisfying all requirements

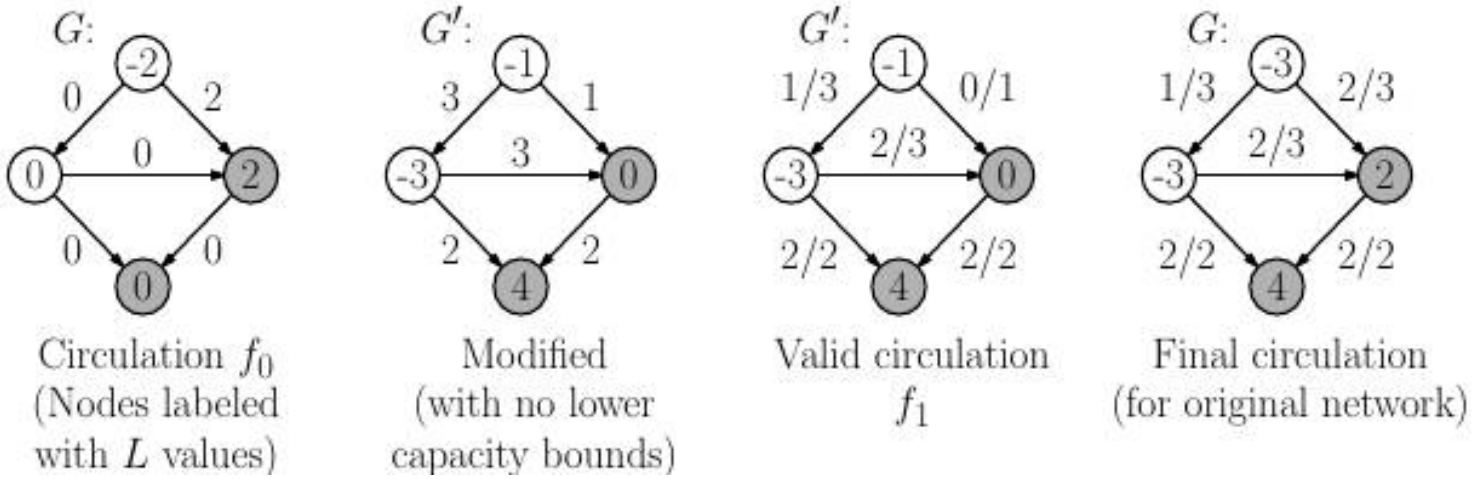


Circulation f_0 (Nodes labeled with L values)

Modified (with no lower capacity bounds)

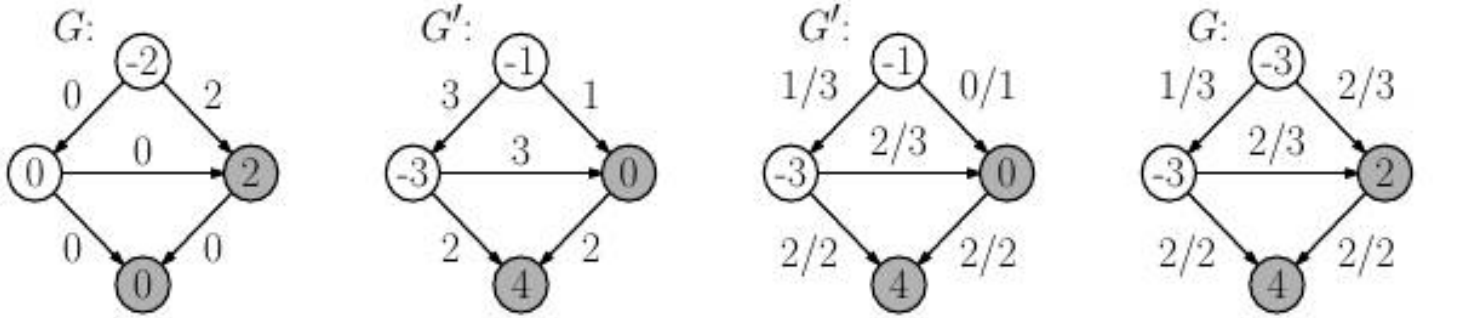
Circulation with Capacity Bounds: Intution (2)

- **How to adjust?** We want to generate a **net flow** of d_v units coming into v and cancel out the excess L_v coming in. That is, we want f^1 to satisfy: $f^1_{in}(v) - f^1_{out}(v) = d_v - L_v$.
- This means that if we combine f^0 and f^1 by summing their flows for each edge, then the final flow into v will be d_v, as desired.
- **How to determine whether there exists such a circulation f^1 ?** set the edge capacities as $c(u, v) - l(u, v)$ as we have already sent $l(u, v)$ units of flow through the edge (u, v). (no need of undoing flow)



Circulation with Capacity Bounds: Reduction (2)

- Create an initial pseudo-circulation f^0 by $f(u,v) = l(u,v)$ (1st Figure)
- Create a new network $G' = (V', E')$ with $V' = V$ and $E' = E$ of G
- For each $(u,v) \in E'$, set **adjusted capacity** to $c'(u,v) = c(u,v) - l(u,v)$
- For each $v \in V'$, set **adjusted demand** to $d'_v = d_v - L_v$, where, $L_v = f^0_{in}(v) - f^0_{out}(v)$ (2nd Figure)
- Invoke any circulation algorithm on G' (3rd Figure).
- If algorithm reports there is **no valid circulation** for G', then there is **no valid circulation** for G. Else if algorithm returns a valid circulation f^1 for G', circulation for G is combination $f^0 + f^1$ (4th Figure)
- **Time complexity: sum** of the time to perform the reduction $O(n+m)$ and time to compute the circulation, i.e time to compute a maximum flow.



Circulation f_0 (Nodes labeled with L values) | Modified (with no lower capacity bounds) | Valid circulation f_1 | Final circulation (for original network)

Table of Contents

1. Preliminaries
2. Ford-Fulkerson Method
3. Analysis of Ford-Fulkerson Method
4. Maximum Flow (Max Flow) and Minimum Cut(Min Cut)
5. Maximum Bipartite Matching : A reduction to Maximum Flow Problem
6. Extension of Network Flows
   - Circulation
   - Circulation with upper and lower capacity bounds
   - Application: Survey design
7. Books

Application: Survey Design

Problem: A company sells k different products, and maintains a database storing which customers have bought which products recently. Design a survey and send it to a subset of n customers. Survey must be sent to appropriate customer following some guidelines to be satisfied. The problem is which product survey is to be send to which customer without bothering them much but company's minimum requirement of feedback will be satisfied.

- The survey sent to a customer will ask questions only about the products this customer has purchased.
- Based on the knowledge of how many products customer i has purchased, marketing people has defined two bounds, c_i and c'_i, the minimum and maximum number of products about which customer i will be asked. Note that $0 \leq c_i \leq c'_i$
- To get a balanced amount of information about each product, marketing people have defined two bounds $0 \leq p_j \leq p'_j$, so that at least p_j and atmost p'_j customers will be asked about product j.

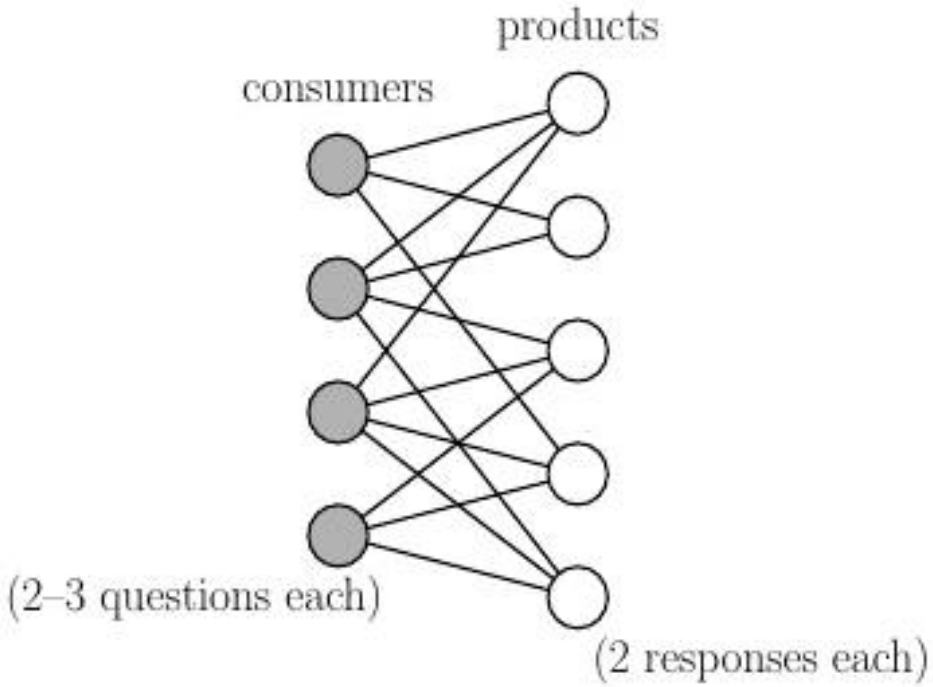
Application: Survey Design Model (1)

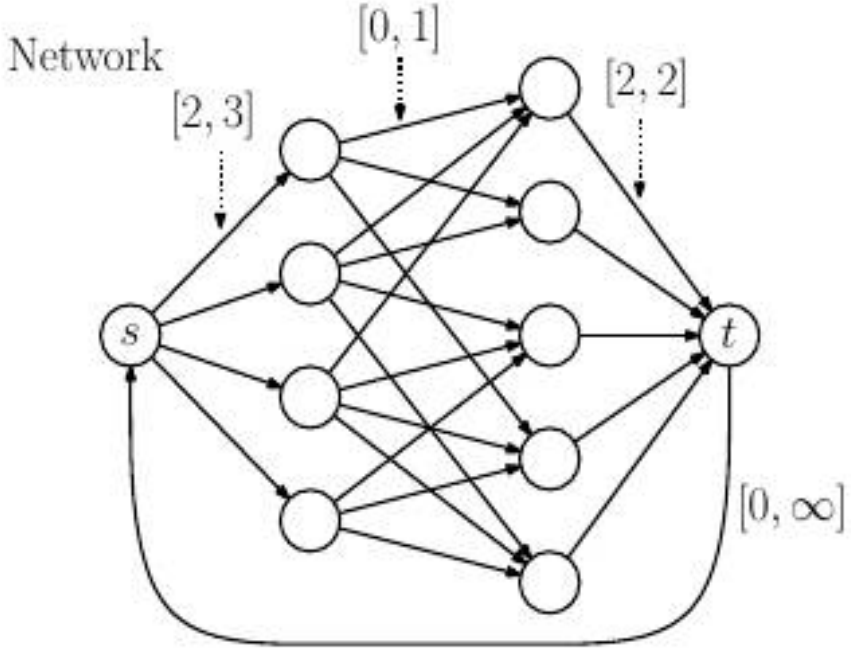
- Create a **Bipartitie graph** $G = (C \cup P, E)$, where C and P are the set of customers and products.
- For each customer i who purchased product j create a directed edge (i, j) with an **upper flow bounds** of 1. This models the requirement that customer i will be surveyed **at most once** about product j, and customers will be asked only about products they **purchased**.
- Create a source vertex s and connect it to all the customers C, where the edge from s to customer i has lower and upper flow bounds of c_i and c_i' , respectively. This models the requirement that customer i will be asked about at least c_i products and at most c_i'.
- Create a sink vertex t, and create edge from product j to t with lower and upper flow bounds of p_j and p_j'. This models the requirement that there are at least p_j and at most p_j' customers will be asked about product j.
- create an edge (t, s). Its bound is $[\sum_i c_i \sum_i c_i']$. This is needed for technical reasons, since we want a circulation.
- All node demands are set to 0.

Application: Survey Design Model (2)

- The flow through each customer node will reflect the number of products this customer is asked about.
- The flow through each product node will reflect the number of customers that are asked about this product.
- The flow carried by the edge (t, s) corresponds to the overall number of questions asked
- If G has a valid (integer valued) circulation, there is a flow of one unit along edge (i, j) if customer i is surveyed about product j.
- From the capacity constraints, customer i receives somewhere between c_i and c_i' products to answer questions about, and, each product j is asked about between p_j and p_j' customers.
- Since the node demands are all 0, the flows through every vertex (including s and t) satisfy flow conservation.
- This implies that the **total number of surveys** sent to all the customers (the flow out of s) equals the total number of surveys received on all the products (the flow into t).

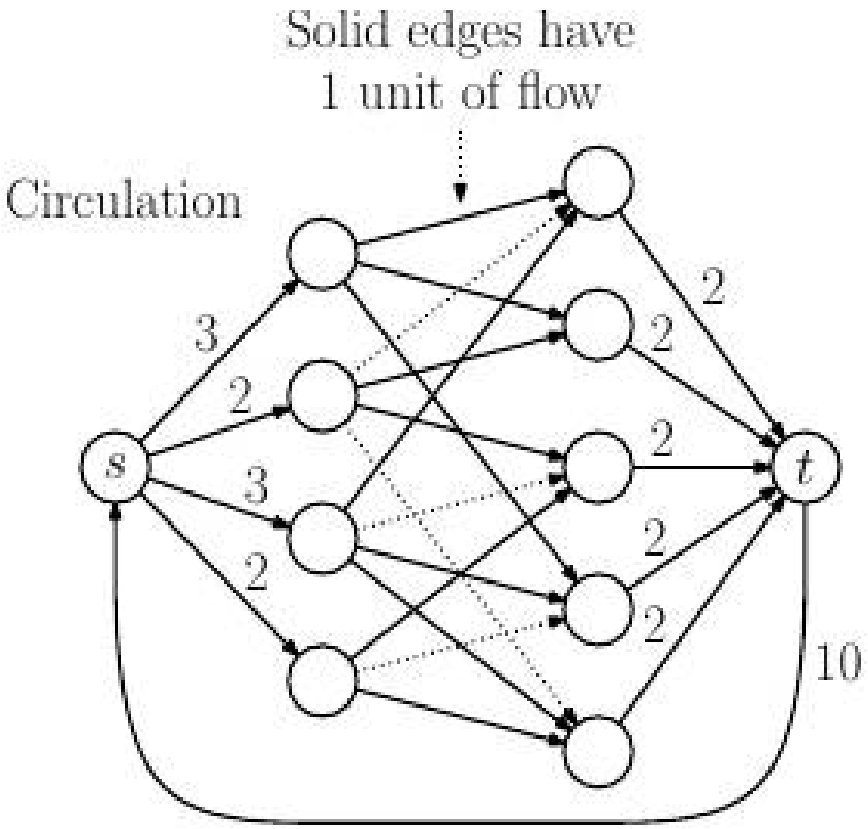
Example Survey Design formulation





Survey Design Example Solution

- The converse is also true, namely, that a valid survey design implies the existence of a circulation in G.
- So, there exists a valid circulation in G' if and only there is a valid survey design.



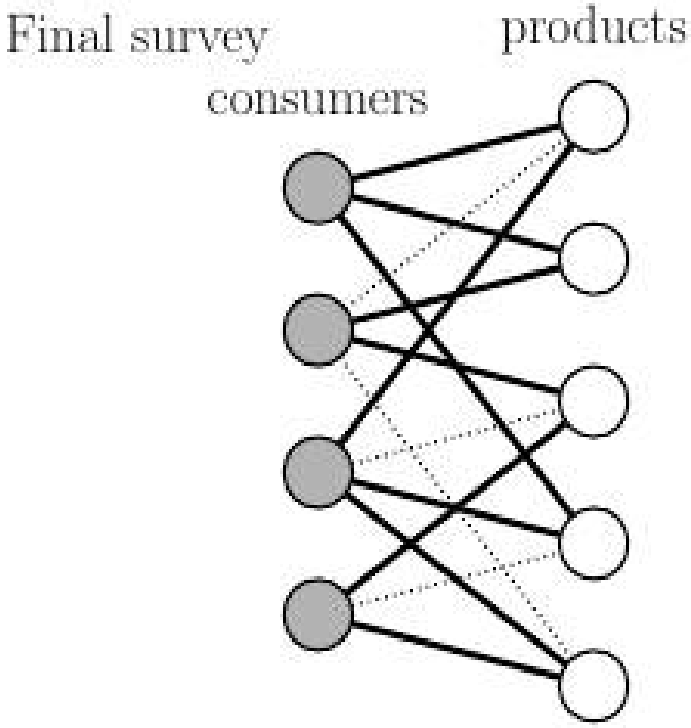


Table of Contents

1. Preliminaries
2. Ford-Fulkerson Method
3. Analysis of Ford-Fulkerson Method
4. Maximum Flow (Max Flow) and Minimum Cut(Min Cut)
5. Maximum Bipartite Matching : A reduction to Maximum Flow Problem
6. Extension of Network Flows
   - Circulation
   - Circulation with upper and lower capacity bounds
   - Application: Survey design
7. Books

References

- Lecture Notes of David Mount, Design and Analysis of Algorithms, CMSC 451, Fall 15, Lecture 18,19,20
- J. Kleinberg and E. Tardos, Algorithm Design, Pearson, Chapter 7.1,7.2,7.3,7.5,7.7,7.8

The End